LaunchDarkly SE Technical Exercise

This exercise is intended to show us how effectively you can learn — and learn how to articulate — a new technology, as well as to provide a baseline expectation of your DevOps-related skillset. We know your time is valuable, and we want you to know we value the quality of the output more than the speed in which you deliver. If this exercise is going to take you longer than a week to deliver, that is OK, please just let us know that you are still working on it and are still interested in our process. **Treat as a sample app you are delivering to a prospective customer.**

The assignment should be completed in two parts. Both components should be completed using one of LaunchDarkly's top 5 most common SDKs:

- Java
- .NET
- Javascript (or any js derivative)
- Python
- Go

How to Submit Your Work

When you're ready, upload your solution to a public GitHub repo, and **be sure to include instructions on how to set up and run the sample implementation**. The instructions need to explain — in detail — how the user can run the example, include any assumptions you are making about their environment.

Be sure to include relevant comments in the code, (e.g. where the SDK key needs to be replaced, or where a feature flag needs to be re-created by the user, etc.).

We hope you're as excited to dig in as we are to see your project. If you are, please go and dig into the LaunchDarkly platform, read our documentation (https://docs.launchdarkly.com/), sign up for a trial account (https://launchdarkly.com/start-trial/) and put together a project that shows us what you can do!

Part 1: Release and Remediate

Scenario:
You are an engineering manager at ABC Company. Your company's competitors seem to be catching up with your company's SaaS solution. As such, executive leadership is pressuring development teams to turn out features faster. One of your company's greatest values is the high quality with which it delivers in software releases. Since quality must remain high, you're tasked with finding a way to deliver features faster without increasing risk. With this in mind, you'll need to figure out a safe way to test your code in production before releasing to your customers. After release, should a bug slip through, you also need a way to quickly rollback the release with minimal to no impact on your customers.

Expectations:

- **Feature Flag:** Implement a flag around a specific new feature. You should be able to demonstrate releasing the feature by toggling the flag on, and rolling it back by toggling it off.
- **Instant releases/rollbacks:** Implement a "listener", such that when the flag is toggled, the application instantly switches to the new/old code (i.e. no page reload required)
- **Remediate:** Use a trigger to turn off a problematic feature (can be done manually, e.g. via curl, or via a browser).

Part 2: Target

Scenario:
You are a developer at ABC Company. Your organization is working on revamping your landing page. It's a project that spans multiple teams, but you and your team are working on a specific component (for this exercise — of your choosing). The web application has about 40,000 visitors on average each day, so any changes and/or issues would significantly impact the user experience. With that said, this project has a lot of eyes on it, and you want to ensure you are shipping well-tested code. Lucky for you: your organization will be leveraging LaunchDarkly for this project, which will allow you to implement individual and rule-based targeting.

Expectations:

- **Feature Flag:** Implement a feature flag around a specific component. This can be the same feature from your "release and remediate" example or a new one.
- **Context Attributes:** Create a context with attributes of your choosing; you will use these attributes to target your feature release.
- **Target:** Demonstrate both individual targeting and rule-based targeting.

Extra Credit: Experimentation

Scenario:
You are a product manager at ABC Company. Your organization has been working on revamping the landing page, and you have been working with your development team to (implement the feature from the Targeting example). You now need to measure the impact of the new feature since you want to help your organization make decisions informed by accurate data.

Expectations:

- **Feature Flag:** Use the same feature flag you already created for the Targeting example.
- **Metrics:** Create a metric.
- **Experiment:** Create an experiment using the feature flag and metric you just created.
- **Measure:** Run the experiment long enough to gather enough data to make an informed decision.

Extra Credit: AI Configs

Scenario:
You are an AI product manager at ABC Company. Your organization is rolling out a chatbot to assist customers, and you have been suggesting improvements to the prompts used by the LLM. You need to manage what models and prompts are used in your GenAI product in order to find the most effective configuration for your users.

Expectations:

- **AI Config:** Implement an AI configuration in your application that allows you to quickly change prompts and models.
- **(Optional) Experiment:** Test variants of prompts and models to see what is most effective based off any metrics you deem important.

Extra Credit: Integrations

As part of any of these scenarios, you may find it interesting or helpful to explore any of LaunchDarkly's many integrations (https://docs.launchdarkly.com/integrations). This is not necessary to produce a top-tier exercise but may offer deeper insight into the product and opportunities for a more compelling demo.